

Job Market - Etape 2 - Rappel sur les données relatives aux offres d'emploi

Cette section rappelle le schéma commun construit à partir des données France Travail et Adzuna.

L'API France Travail fournit des données plus riches que l'API Adzuna. **Certains champs proposés par France Travail seront donc maintenus**, quitte à obtenir des valeurs "null" sur ces champs pour les modalités transmises par Adzuna.

Liste des champs principaux de France Travail maintenus après normalisation, malgré l'absence d'équivalence des champs Adzuna :

- **romeCode** : nomenclature de France Travail destinée à classer les métiers et les emplois. La première lettre du code ROME fournit le code du grand domaine de l'offre d'emploi, par exemple :
 - A : Agriculture / Pêche / Espaces verts et naturels / Soins aux animaux
 - B : Arts / Artisanat d'art
 - C : Banque / Assurance
 - C15 : Immobilier
 - ...
- **codeNAF** : (Nomenclature d'activités française) ce code est attribué par l'INSEE à une entreprise ou à un établissement selon son *activité principale*.
- **competences** et **formations** sont également maintenus dans les champs qui nous intéressent, afin d'étoffer les informations dans le futur dashboard du projet.

1 Tableau final de mapping

Les données normalisées devront présenter un champ **source** qui indiquera soit France Travail soit Adzuna.

Le tableau suivant reprend le schéma normalisé proposé et précise, pour chaque champ, son type ainsi que sa correspondance dans les deux sources.

Champ normalisé	Type final	France Travail (champ / type)	Adzuna (champ / type)
source	str	France Travail (str)	Adzuna (str)
source_id	str	id (str)	id (str)
title	str	intitule (str)	title (str)
company	str	entreprise.nom (str)	company.display_name (str)
description	str	description (str)	description (str)
creationDate	str	dateCreation (str)	created (str)
contractType	str	typeContrat (str)	contract_type (str)
contractTime	str	dureeTravailLibelleConverti (str)	contract_time (str)
locationZipCode	str	lieuTravail.codePostal (str)	—
locationDepartment	str	lieuTravail.libelle (str)	location.area[2] (list[str])
locationCity	str	lieuTravail.commune (str)	location.area[3] (list[str])
locationLatitude	float	lieuTravail.latitude (float)	latitude (float)
locationLongitude	float	lieuTravail.longitude (float)	longitude (float)
salary	float	salaire.libelle (str)	salary_min, salary_max (int / float)
url	str	origineOffre.urlOrigine (str)	redirect_url (str)
codeNAF	str	codeNAF (str)	—
romeCode	str	romeCode (str)	—
category	str	romeCode (str)	category.tag (str)
categoryLabel	str	romeLibelle (str)	category.tag (str)
skills	list[str]	competences[].libelle (str)	—
education	str	formations[].niveauLibelle (str)	—
educationField	str	formations[].domaineLibelle (str)	—

2 Champs nécessitant une transformation des données

Les champs ayant besoin d'être processés avant intégration dans une BDD normalisée sont notamment :

2.1 API France Travail

- `lieuTravail.libelle` -> `locationDepartment` et `locationCity` : Le département (en chiffres) et le nom de la ville doivent être extraits d'une chaîne de caractère de la forme "34 - Montpellier". Cette construction du libellé est **uniforme** sur l'ensemble des offres d'emplois.
- `salaire.libelle` -> `salary` : le libellé fourni par France Travail est une chaîne de caractères saisie par l'API, mais dont le salaire doit être extrait.
 - Idée : extraire le(s) salaire(s) de la chaîne de caractères grâce à leur format uniforme = 4 chiffres, 1 point, 1 chiffre.
 - Faire la moyenne des résultats obtenus, car il est parfois indiqué une fourchette de salaires.

2.2 API Adzuna

- `location.area` -> `locationDepartment` : Le département se trouve de manière **uniforme** en 3ème position dans la liste du champ. On prend donc `locationRegion = location.area[2]`. Il faudra ensuite lui faire correspondre le **numéro du département**, ce qui est plus robuste que le nom en toutes lettres.
- `location.area` -> `locationCity` : Lorsqu'elle est indiquée, la ville se trouve de manière **uniforme** en 4ème position dans la liste du champ. On prend donc `locationCity = location.area[3]` en prenant garde de **traiter le cas où aucune ville n'est renseignée**.
- `salaire_min` et `salaire_max` -> `salary` : calculer la moyenne des 2 champs.

2.3 Les catégories d'offres d'emploi

La taxonomie commune du projet **Job Market** reprend les domaines et sous-domaines du référentiel **ROME de France Travail**.

L'objectif est de conserver la précision disponible chez France Travail :

- pour **France Travail**, `category` est déterminé à partir de `romeCode` ;
- pour **Adzuna**, `category.tag` est traduit vers la catégorie ROME la plus proche ;
 - `categoryLabel` contient le libellé lisible associé à `category` ;
 - les catégories absentes, trop générales ou inconnues sont classées dans **UNKNOWN**.

Ainsi, un `romeCode` France Travail égal à M1805 donnerait le format suivant :

```
{
  "romeCode": "M1805",
  "category": "M18",
  "categoryLabel": "Informatique / Télécommunication"
}
```

Lorsqu'aucun sous-domaine spécifique du mapping n'est défini, seule la lettre du grand domaine est utilisée. Par exemple, un code F1106 est classé en F.

Concernant les catégories fournies par l'API Adzuna, la liste des **29 catégories** (30 catégories si l'on considère "unknown") a été récupérée via l'API Adzuna `/jobs/{country}/categories` qui *"list available categories"*.

On obtient le tableau de mapping suivant pour les catégories d'offres d'emploi :

category	categoryLabel	category.tag Adzuna associés
A	Agriculture / Pêche / Espaces verts et naturels / Soins aux animaux	—
B	Arts / Artisanat d'art	creative-design-jobs
C	Banque / Assurance	—
C15	Immobilier	property-jobs
D	Commerce / Vente	sales-jobs, retail-jobs
E	Communication / Multimédia	—
F	Bâtiment / Travaux Publics	trade-construction-jobs

category	categoryLabel	category.tag Adzuna associés
G	Hôtellerie - Restauration / Tourisme / Animation	travel-jobs, hospitality-catering-jobs
H	Industrie	manufacturing-jobs, energy-oil-gas-jobs
I	Installation / Maintenance	maintenance-jobs
J	Santé	healthcare-nursing-jobs
K	Services à la personne / à la collectivité	social-work-jobs, charity-voluntary-jobs, domestic-help-cleaning-jobs, teaching-jobs
L	Spectacle	—
L14	Sport	—
M	Achats / Comptabilité / Gestion	accounting-finance-jobs, admin-jobs
M13	Direction d'entreprise	—
M14	Conseil / Études	consultancy-jobs, engineering-jobs, scientific-qa-jobs, customer-services-jobs, legal-jobs
M15	Ressources Humaines	hr-jobs
M16	Secrétariat / Assistanat	—
M17	Marketing / Stratégie commerciale	pr-advertising-marketing-jobs
M18	Informatique / Télécommunication	it-jobs
N	Transport / Logistique	logistics-warehouse-jobs
UNKNOWN	Non classé / Domaine inconnu	other-general-jobs, graduate-jobs, part-time-jobs, catégorie absente ou tag inconnu

BDD relationnelle - Données sur les entreprises

La base PostgreSQL sera utilisée pour compléter les informations contenues dans la base MongoDB, en stockant des **informations structurées relatives aux entreprises** identifiées dans les offres d'emploi.

Contrairement aux offres, stockées sous forme de documents dans MongoDB, les informations relatives aux entreprises sont regroupées dans une table relationnelle **companies**.

1 Table companies

Champ	Description
id	Identifiant de l'entreprise — clé primaire
name	Nom officiel de l'entreprise
name_normalized	Nom normalisé utilisé pour le rapprochement avec les données MongoDB
siren	Numéro SIREN de l'entreprise
headquarters_address	Adresse du siège social
creation_date	Date de création de l'entreprise
logo_url	URL permettant d'accéder au logo de l'entreprise
employee_count	Effectif salarié, <i>donnée destinée à être actualisée périodiquement</i>

2 Liaison entre MongoDB et PostgreSQL

MongoDB et PostgreSQL ne disposent pas d'une contrainte de clé étrangère entre eux.

Le rapprochement est donc réalisé à partir d'un **nom d'entreprise normalisé** :

- MongoDB → `offers.nameNormalized`
- PostgreSQL → `companies.name_normalized`

Le nom normalisé est notamment :

- converti en minuscules ;
- débarrassé de ses accents ;
- nettoyé de certains caractères spéciaux ;
- débarrassé des espaces multiples.

Le nom original de l'entreprise reste conservé pour l'affichage.

Cette solution permet de conserver une architecture relativement simple tout en limitant les problèmes liés aux différences de casse ou d'accentuation entre les différentes sources de données.

Diagramme UML

Voir fichier joint.

La relation représentée dans le diagramme n'est pas une clé étrangère au sens du langage SQL, les données étant stockées dans deux systèmes de gestion de bases de données distincts.

Justification du choix des SGBD

MongoDB

MongoDB est utilisé pour stocker les offres d'emploi collectées depuis 2 sources.

Le format documentaire est particulièrement adapté à ces données, les API renvoyant des fichiers JSON. De plus, les données peuvent potentiellement évoluer dans le temps et présenter des différences de structure selon leur provenance.

Il permet également de conserver facilement **une offre** sous la forme d'**un document unique**, proche du format JSON utilisé lors de la collecte et de la normalisation des données.

PostgreSQL

PostgreSQL est utilisé pour stocker les informations relatives aux entreprises.

Ces données sont plus structurées, stables et se prêtent bien à une organisation relationnelle.

L'utilisation d'une table `companies` permet notamment de définir clairement les types de données et les informations obtenues par *webscraping* sur le site <https://annuaire-entreprises.data.gouv.fr/>.

PostgreSQL est ainsi adapté pour enrichir les offres stockées dans MongoDB.